

# LLD Case Study 1: Distributed Job Scheduler

---

**Target Patterns:** Strategy Pattern, Command Pattern, Observer Pattern

**Key Engineering Focus:** Thread-safe task execution, pluggable scheduling algorithms, worker coordination, and lifecycle event streaming.

---

## 1. Problem Statement & Functional Requirements

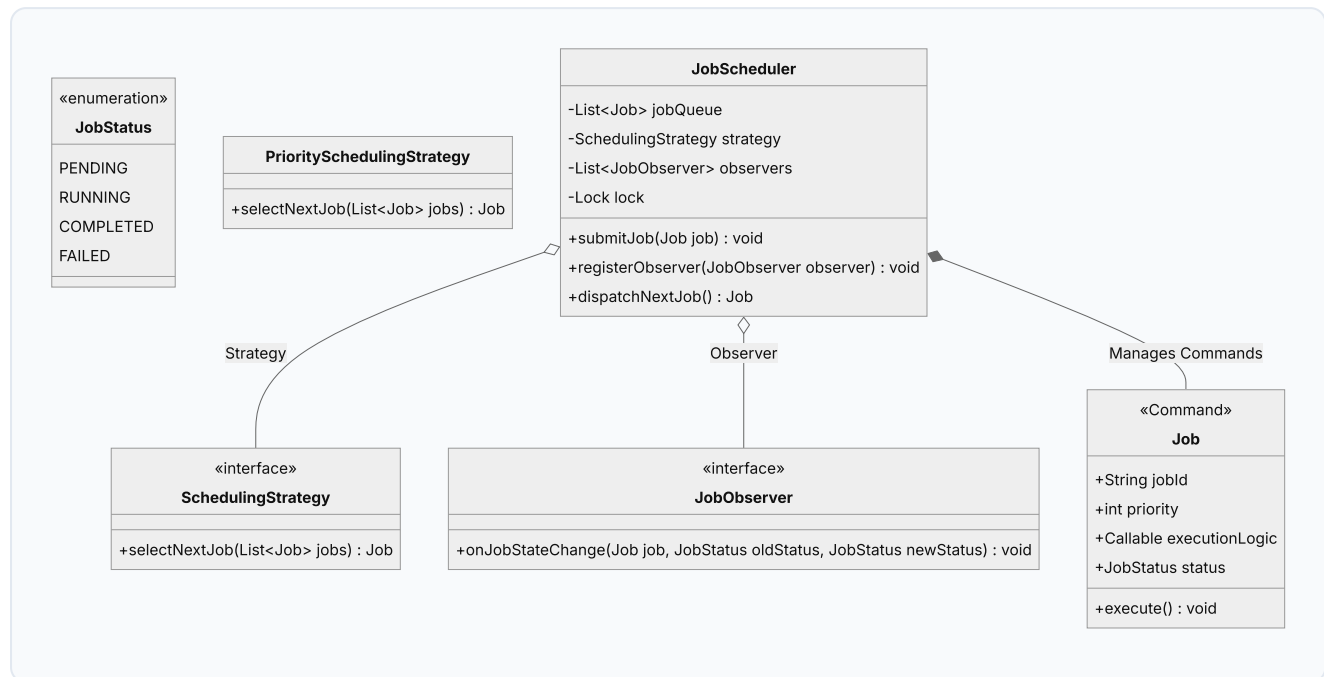
---

Design a modular in-process Job Scheduler capable of scheduling, queuing, executing, and monitoring tasks across worker pools.

### Requirements:

- Job Definition (Command Pattern):** Encapsulate jobs as executable units of work with retry counts and parameters.
  - Scheduling Policy (Strategy Pattern):** Support pluggable scheduling algorithms (e.g. First-Come-First-Served / Priority Scheduling vs Shortest Job First).
  - Lifecycle Notifications (Observer Pattern):** Allow external monitoring systems (logging, metrics, alerting) to subscribe to job state changes ( `PENDING` , `RUNNING` , `COMPLETED` , `FAILED` ).
  - Thread-Safety:** Multiple threads must be able to submit and execute jobs concurrently without state corruption.
-

## 2. Architecture & UML Class Diagram



## 3. Production-Grade Python Implementation